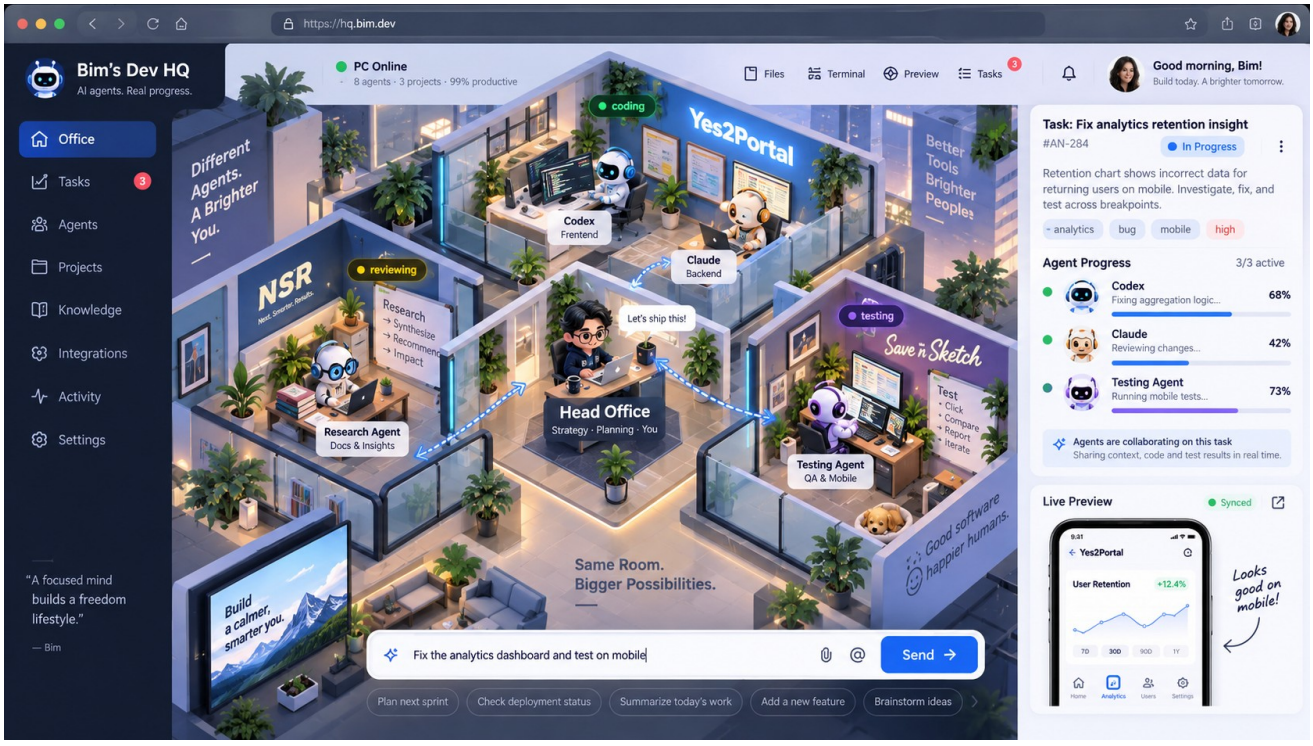
PERSONAL AGENT WORKSPACE

Technical Blueprint v0.7

Execution Baseline - Contracts Closed, External Gates Explicit

A single-owner, local-first control plane for working from anywhere with AI agents, domain-aware review, recoverable mechanics, and an office view derived only from durable truth.

v0.7 rule
Build for one owner, one Mac Mini, one domain today. Pay only for deliberate seams. Close the contracts that Phase 0 depends on; leave external Phase -1 proofs explicit rather than pretending they are document decisions.



Early visual concept. Branding shown inside the artwork is legacy and non-binding.

Version: 0.7 • Status: execution baseline for Phase -1 / Phase 0 • Product naming: descriptive placeholder until Phase -1 clearance

00 Executive Summary

v0.7 closes the contract and data-model blockers identified in the v0.6 review. The system remains single-owner, single-host, local-first and phone-friendly. Portability is still purchased only at deliberate seams, but the core now admits a small versioned set of change-render shapes, applies privileged operations through declarative plans, exposes sandbox dirtiness before cleanup, and defines review-readiness, repair exits, session auditability and liveness semantics explicitly.

Execution status: document-level Phase 0 blockers are resolved here. Phase -1 still has real-world gates - naming clearance, canonical-origin proof and real cost ceilings - that must be completed before the corresponding implementation gates are crossed.

The one rule

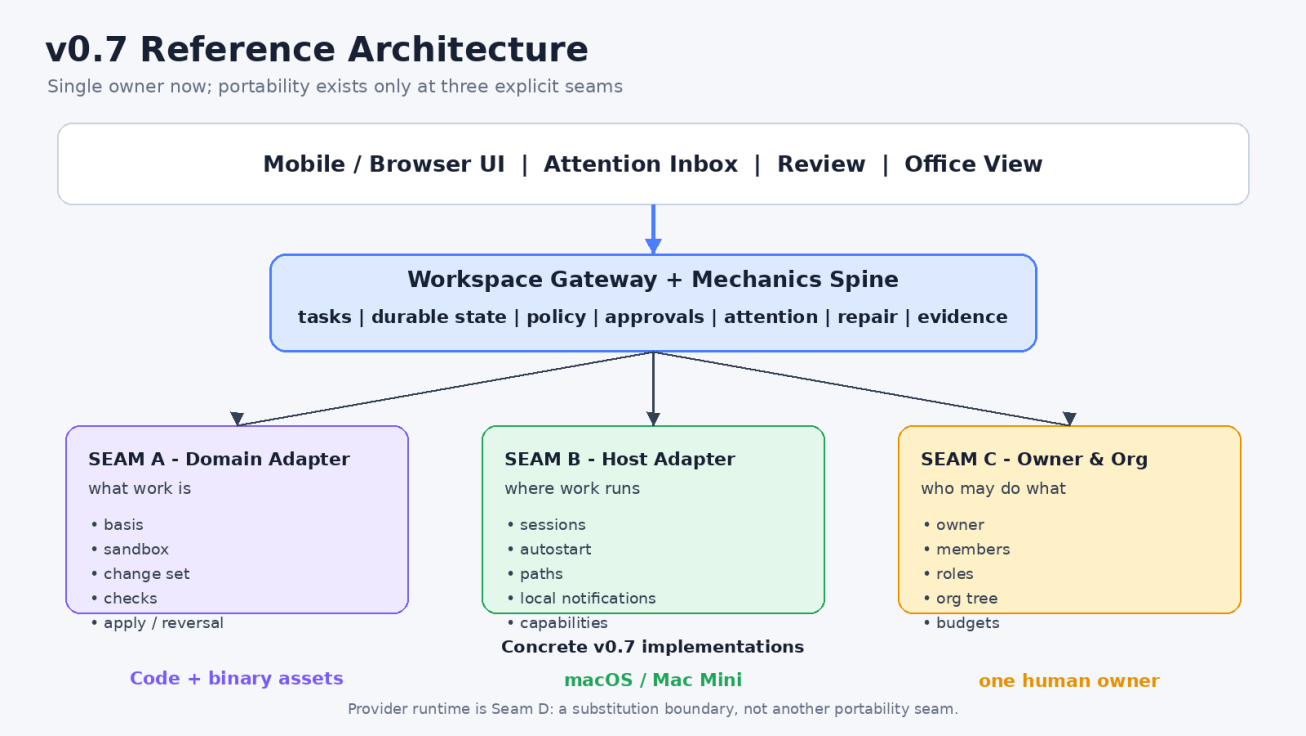
Build for one owner, one Mac Mini, one domain. Pay only for the seams. A seam costs a type boundary and a naming discipline, not multiple implementations on day one.

What remains non-negotiable

- Never animate a lie: the office renders only from snapshots plus durable events.
- Durable versus ephemeral event split: PTY bytes, token deltas and progress noise are never durable state.
- needs-repair is a real state: ambiguity freezes mutation instead of guessing.
- Operator exits exist for unresolved steering and repair cases.
- Apply/landing authority fails closed: AI judgment does not own irreversible consequences.
- Evidence is part of task output, not a byproduct.
- One authoritative home per fact.
- Attention cost matters more than agent concurrency.
- Waiting is tokenless and event-driven.

Three portability seams + one substitution seam

Boundary	What varies	v0.7 implementation
A - Domain adapter	Basis, sandbox, change set, checks, apply planning/reconcile	Code reference + read-only asset slice before trust UI freezes
B - Host adapter	Session lifetime, autostart, path rules, local notifications, host capabilities	macOS / Mac Mini only
C - Owner & org	Identity, roles, org structure, capability policy, budget hierarchy	One human owner; roles/org nodes are data
D - Provider substitution	Codex/Claude session semantics, events, steering, usage	Real provider integrations remain specific; unify only after two implementations



v0.7 architecture: one thin spine with only three portability seams.

Primary v0.7 outcome

A single owner can leave the Mac Mini running, inspect agent work and attention state from a phone, review a small change set with evidence, resolve ambiguity from the phone, and authorize an apply operation with explicit reversibility information. The same mechanics must work for code and one genuinely non-text asset workflow before the domain contract is considered stable.

01 Contents

02 Product Vision, North Star and Scope Discipline

2.1 The product

Workspace is a personal operating surface for a development machine that keeps working when the owner is away from the keyboard. It exposes attention, terminal sessions, previews, tasks, agent activity, evidence and controlled apply operations through a browser and phone-friendly UI. The office metaphor is a visualization layer over real durable state, not a simulation.

2.2 North-star metrics

Metric	Why it matters	Direction
Time from task ready to confidently landed	Measures review burden rather than worker throughput	Down
Median owner attention per landed task	The true bottleneck is human review bandwidth	Down
Rework / rollback / escaped-defect rate	Prevents approval-count gaming	Flat or down
Unknown/needs-repair time to recovery	Measures whether fail-closed states are usable from mobile	Down
Waiting token spend	Idle supervision must be free	Near zero

2.3 Scope rule: pay only for seams

> **Design test**
> The test is not “does v0.7 support future X?” The test is “when X becomes real, do we add a module/configuration, or edit thirty files?”

- The UI may be opinionated and single-owner.
- The gateway may be one process and SQLite may be the only database.
- The host implementation may be unapologetically macOS-specific.
- The code adapter may use git deeply - but git language must not leak into the core protocol or schema.
- The asset adapter may use Unity/Defold-specific concepts behind the domain seam.

2.4 Core vocabulary renames

v0.6 core term	v0.7 core term	Where old/domain term remains valid
planned_against_commit	basis.ref inside Basis	Code adapter: commit SHA
context_inputs[path,sha]	basis.inputs[resource_id,version]	File paths are one resource type
Worktree	Sandbox	Code adapter may implement with git worktree
Diff / hunk	Change set / change anchor	Text render shape may contain hunks
diff_budget_lines	change_budget + change_unit	Code unit=lines/files;

		assets=assets/MB
Merge / push / land	Apply	“Land” remains user-facing copy
Repo / project root	Source of record	Code source may be a repository
Tests / lint / typecheck	Checks	Declared per project/domain
Build lock	Exclusive resource lock	Unity build lock is one named lock

03 Build-vs-Buy and Phase -1

3.1 Revisited decision

v0.6 leaned toward reusing an existing code orchestrator as the mechanics source of truth. v0.7 partially inverts that decision because the second required domain is binary creative assets. Code-oriented orchestrators are useful substrates inside the code adapter, but they cannot define the whole system without re-importing git-shaped assumptions.

Own the thin spine

Own org/role, task, basis, sandbox lifecycle, change set, checks, apply authority, attention, repair, evidence and review threads. Delegate underneath that spine where a mature tool is a good fit.

3.2 Where existing tools still fit

If the Phase -1 ADR delegates any code-domain lifecycle to an external orchestrator, the code adapter must maintain an explicit projection and emit adapter.divergence whenever substrate and spine disagree. Divergence count/age are monitored and target zero; unknown divergence fails closed rather than allowing the office to animate projected state as truth.

Layer	Build / buy stance	Examples to evaluate
Code-domain mechanics	Prefer delegation if contract maps cleanly	hand; DevHQ.app-class worktree/review tooling; Firstmate-style supervision
Agent runtime/session	Delegate heavily	Codex / Claude native sessions; terminal session manager
Persistent terminal session	Buy/reuse	tmux first; shpool/atch candidates
Remote networking	Buy/reuse	Tailscale primary; Cloudflare fallback path
Workspace spine + cross-domain review/attention	Own	This is the unique product

3.3 Phase -1: trial + probe + external gates

1. Drive one existing orchestrator on real code work for two weeks. Log every place its data model cannot express a non-code unit of work.
2. Run one real Unity/asset task manually but instrument it: identify basis, sandbox, change set, checks, evidence, ownership and reversibility.
3. Prove one canonical browser origin over both private and fallback network paths with a throwaway hostname before registering a real passkey RP ID.
4. Complete naming clearance for repo, CLI, hostname and WebAuthn origin before Phase 0 writes persistent identifiers.

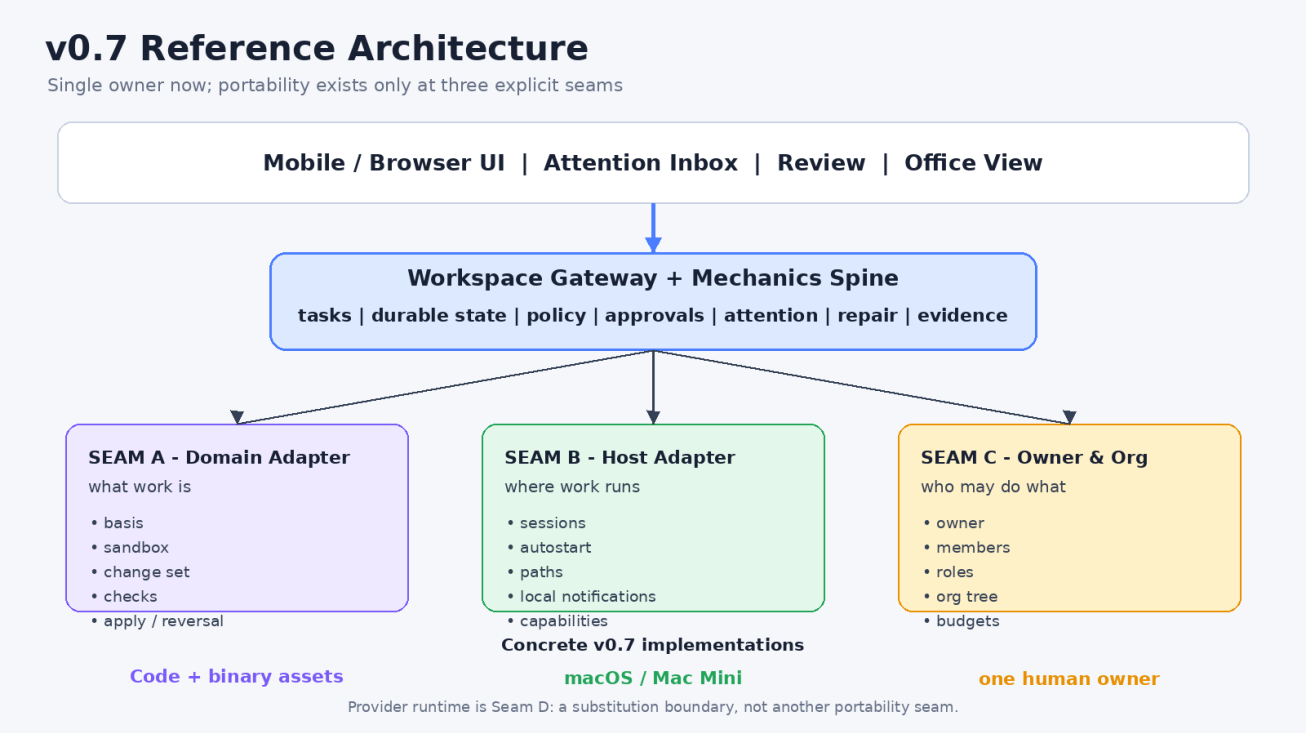
3.4 Phase -1 ADR output

The ADR chooses: the thin spine scope; whether the code adapter delegates to a specific orchestrator; whether the asset adapter is one adapter with capability probes or a family; the canonical origin approach; and the cleared product/CLI naming. The v0.6 four-week calendar is intentionally deleted. Phases are evidence gates, not dates.

Stop branch still exists

If the trial shows an existing orchestrator plus a thin mobile/review layer solves the daily workflow and the asset requirement is not real, stop before generalizing. The seams prevent lock-in; they do not force scope.

04 Reference Architecture and Trust Boundaries



4.1 One gateway, one authority

The browser is never granted generic “control my Mac” authority. A single local gateway exposes typed operations. It owns durable application state, policy evaluation, approval semantics, attention state and the adapter bindings. Static React assets may be served by the same gateway.

4.2 Trust boundaries

Boundary	Allowed	Not allowed
Browser -> gateway	Typed API, terminal attach tokens, review decisions, step-up assertions	Arbitrary shell/filesystem credentials
AI -> mechanics spine	Propose tasks, steer, inspect, run adapter-declared operations within capability	Self-grant privileges, bypass stale basis, apply irreversible work
Mechanics -> domain adapter	Invoke contract with policy-scoped workspace/sandbox	Assume git semantics in core
Mechanics -> host adapter	Attach sessions, path policy, health probes, local notifications	Hard-code launchd/tmux/TCC in core
Credential broker -> external systems	Perform approved authenticated action	Expose raw long-lived secret to worker by default

4.3 Truth ownership

- Durable state: SQLite is the authoritative control-plane history.
- Live terminal/token streams: session backend and memory buffers are authoritative; they are not replayed as durable facts.
- Workspace contents: the domain adapter / source of record is authoritative.

- Office visualization: derived from snapshot plus durable events only.
- When reconciliation cannot prove safe convergence: enter needs-repair.

05 Seam A - Domain Adapter

5.1 Basis fingerprint

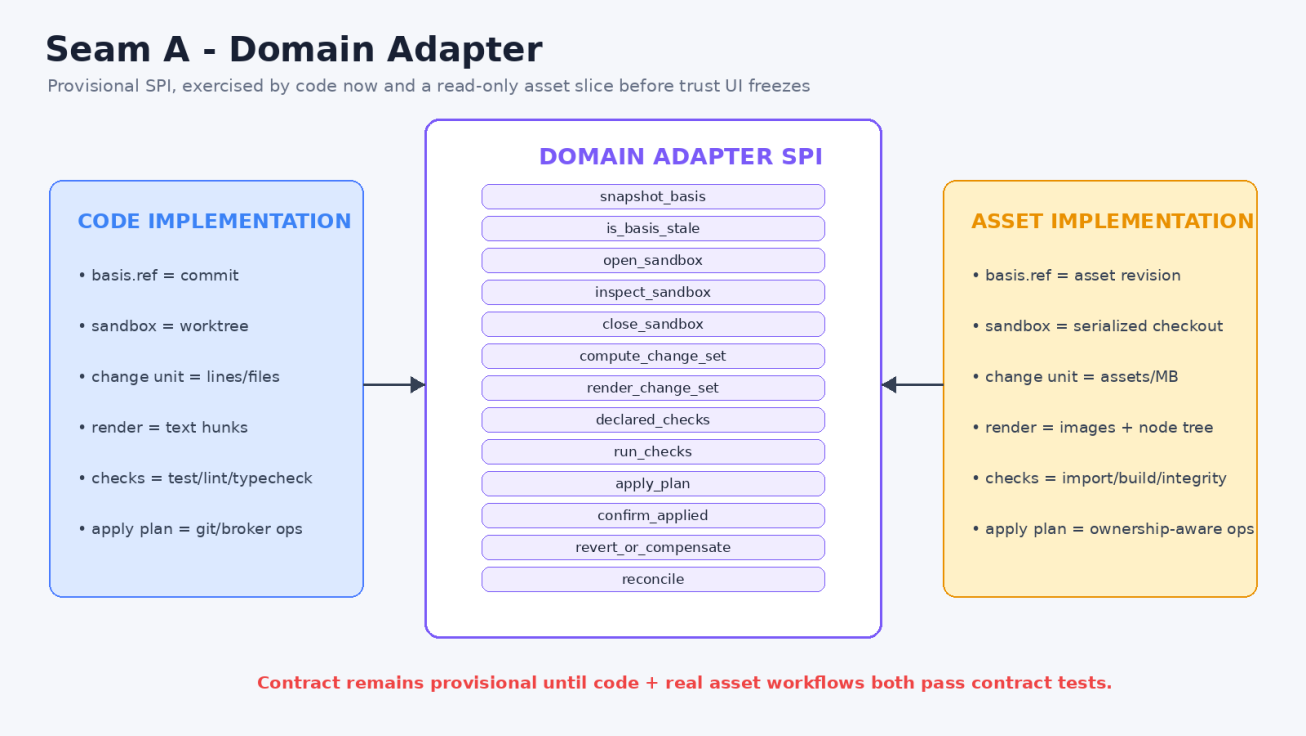
```
basis:
  ref: <opaque version of source of record>
  inputs:
    - resource_id: <consulted resource id>
      version: <hash | etag | mtime+size | revision>
  captured_at: <ts>
```

Only resources actually consulted while preparing the brief are fingerprinted. The basis is checked immediately before dispatch and again after long queue waits. Staleness may be fresh, stale(reason), or unknown. Unknown fails closed.

5.2 Provisional adapter contract

```
DomainAdapter (v0.7 provisional SPI):
  snapshot_basis(project, consulted_inputs) -> Basis
  is_basis_stale(basis) -> fresh | stale(reason) | unknown
  open_sandbox(project, basis) -> Sandbox
  inspect_sandbox(sandbox) -> SandboxInspection
  close_sandbox(sandbox, policy) -> SafetyRecord
  compute_change_set(sandbox) -> ChangeSet
  render_change_set(changeset, surface) -> RenderableChange
  declared_checks(project) -> [CheckSpec]
  run_checks(sandbox, [CheckSpec]) -> [CheckResult]
  apply_plan(changeset, policy) -> ApplyPlan
  confirm_applied(plan_id, broker_results) -> ApplyResult
  revert_or_compensate(apply_result) -> ReversalPlan | not_reversible
  reconcile(known_state) -> converged(state) | needs_repair(probes)
```

The protocol decision is explicit in v0.7: packages/protocol owns a closed, versioned union of RenderableChange and ChangeAnchor shapes. Core therefore knows a small set of review shapes (text_range, asset_id, node_path, region and their render payloads), but not how adapters produce them. The SPI remains provisional until the read-only asset slice exercises basis, inspection, change-set production and rendering; privileged apply stays split into declarative ApplyPlan plus spine-owned broker execution.



5.2.1 RenderableChange and ChangeAnchor protocol

RenderableChange is a closed union in packages/protocol. v1 shapes are: text_patch (files + text ranges/hunks), asset_delta (asset IDs + before/after artifacts), node_tree_delta (node paths + structured property changes), and region_delta (image/scene region references). New shapes require a protocol version bump and a renderer case; adapters do not ship browser code in v0.7.

5.2.2 Apply planning preserves the credential boundary

Adapters never receive credential-broker handles. apply_plan() returns ordered declarative operations, each with target_ref, required capability, risk and reversibility class. The spine validates the plan against the role, current basis, approval fingerprint and budget, then invokes the credential broker itself. confirm_applied() lets the adapter reconcile domain state after execution.

5.2.3 Sandbox inspection before teardown

inspect_sandbox() is non-mutating and may be called at any time. It returns dirty, uncommitted_summary, retained_artifacts and safe_to_close. Cleanup policy and the office dirty-work badge consume this result before close_sandbox() is even considered.

5.3 Code adapter

Core concept	Code implementation
Source of record	Git repository / checked-out code project
Basis ref	Commit SHA

Sandbox	Git worktree or delegated orchestrator work unit
Change set	File changes + text patches
Change anchor	File + range/hunk identifier
Checks	Tests, lint, typecheck, build, screenshot
Apply	Patch/merge/push through broker/policy
Reversal	Revert or follow-up commit where appropriate

5.4 Binary creative-asset adapter

Text assumption to break	Asset-domain requirement
Lines are the unit	Files/assets, byte size, asset count or scene nodes
Change set renders as hunks	Before/after images, asset lists, node trees, metadata deltas
Anchors are lines	Asset IDs, GUIDs, node paths or regions
Parallel sandboxes are cheap	Often serialized checkout plus named exclusive lock
Checks are tests/lint	Import succeeds, build succeeds, reference integrity, screenshot/render
Merge is solved	Ownership or last-writer semantics; no text merge
Evidence can be strong	Manual-required is often the permanent normal case

5.5 Reversibility class

Class	Meaning	Examples
revertible	Clean automated reversal exists	commit, local change, generated artifact
compensable	Undo requires a new forward action	deployment rollback, republish previous asset
irreversible	No reliable reversal	store submission, release publication, sent message, destructive delete

Rule
Irreversible apply always requires fresh per-action user verification regardless of ordinary risk tier. The approval card shows the reversal plan - or the absence of one - before the decision.

06 Seam B - Host Adapter

6.1 macOS-only implementation

The implementation is intentionally Darwin/Mac Mini specific. Portability exists because the host surface is narrow and core packages never import macOS terminology or APIs directly.

```
HostAdapter:
  session_manager() -> attach / reattach / list
  autostart_contract() -> install / status / repair
  path_policy() -> canonicalize / allow-roots
  notify_local() -> local attention surface
  health_probes() -> permissions / power / disk / unlock state
  capabilities() -> { egress_enforcement, ... }
```

6.2 Session lifetime belongs below the gateway

Persistent terminals are owned by tmux first (or a later session manager such as shpool/atch), not by node-pty children of the gateway. The gateway attaches and renders. Restarting the gateway must not terminate the agent session. This is a Phase 0 proof, not a future reliability enhancement.

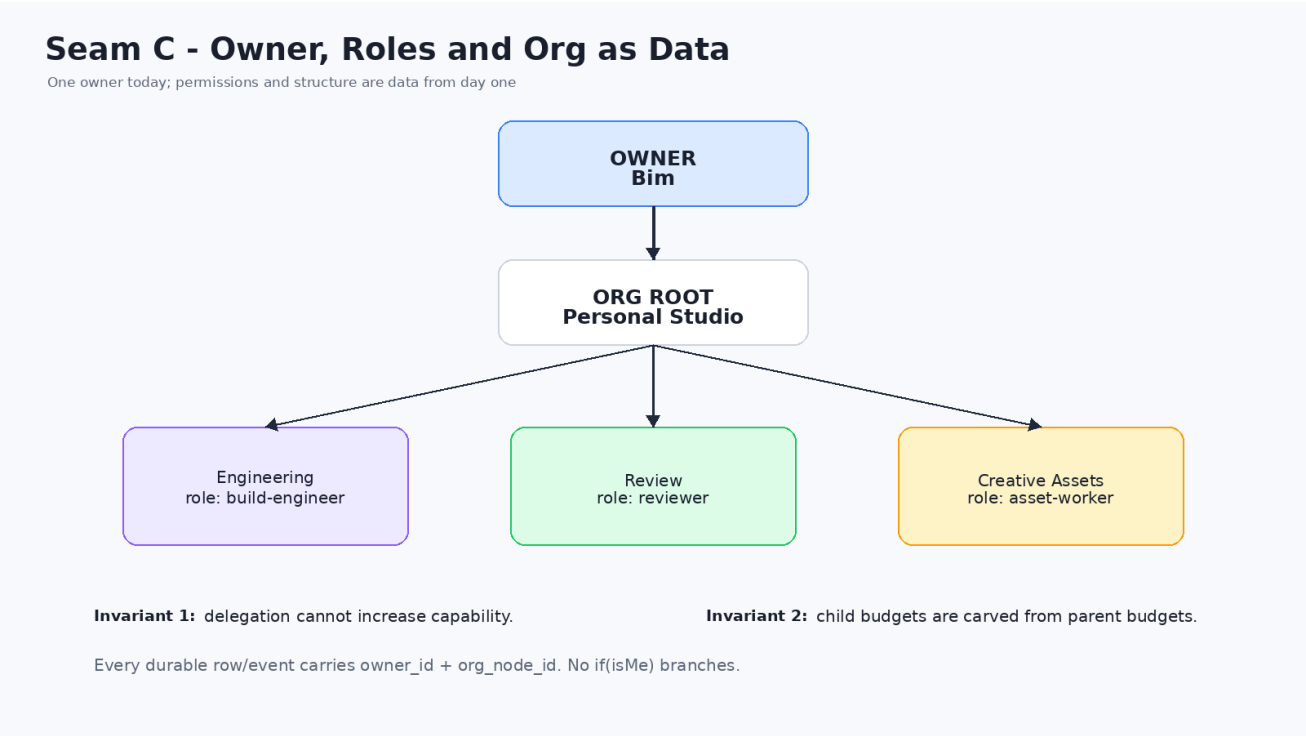
6.3 Host capabilities

The macOS adapter reports egress_enforcement = advisory. The policy layer reads host capabilities instead of globally hard-coding the guarantee. A future isolated Linux runner could report enforced without changing the domain/protocol layer.

6.4 Darwin-specific notes belong in an appendix

- LaunchAgent/autostart and active-login requirements.
- FileVault power-loss availability gap: encrypted boot may require physical unlock.
- TCC / Full Disk Access / path permissions must be surfaced by health probes.
- Power/sleep configuration and disk-space thresholds are local host concerns.

07 Seam C - Owner, Roles and Org



7.1 Minimal identity model

Entity	v0.7 requirement
Owner	id, display, credential references, budget ceilings
Member	id, owner_id, kind human|agent, role_ref; exactly one human row initially
Role	Versioned data template describing charter, capabilities, context, output and limits
OrgNode	Tree node: org | department | desk, with optional policy overrides

Every durable row and durable event carries owner_id and org_node_id from database version 1. Authorization always asks a capability table even though there is only one human owner.

> **Hard rule**
> No if (isOwner) / if (isMe) authorization branches in the codebase. The one-owner case is represented by data, not special control flow.

7.2 Role template

Configuration precedence: the Project establishes domain invariants and the safety floor. change_unit is owned by the Project/domain and a Role cannot override it. For change_budget and evidence requirements, most-restrictive-wins; a Role may tighten but never loosen the Project. A looser Role declaration is a configuration error at load time, not a silent override.

```
role:
  name: build-engineer
  charter: |
    Plain-language purpose and explicit boundaries.
  org_node: studio/engineering
  domain: code
  capabilities: [read_workspace, apply_patch_in_sandbox, run_declared_command]
  connectors: []
  context_packs: [conventions, commands, gotchas]
  output_contract:
    change_unit: lines
    change_budget: 250
    evidence_profile: strong
  review:
    execution_class: standard
    reviewer: fresh_peer
  limits: { cost_per_task, concurrency, escalate_after }
  delegation: []
  runtime:
    provider: codex
    session_policy: persistent
```

7.3 Delegation invariants

5. No privilege escalation by delegation: a role cannot create/task a role whose capability set exceeds its own effective capabilities.
6. Budget is sub-allocated, never additive: child/org-node ceilings are carved from the parent remaining budget.

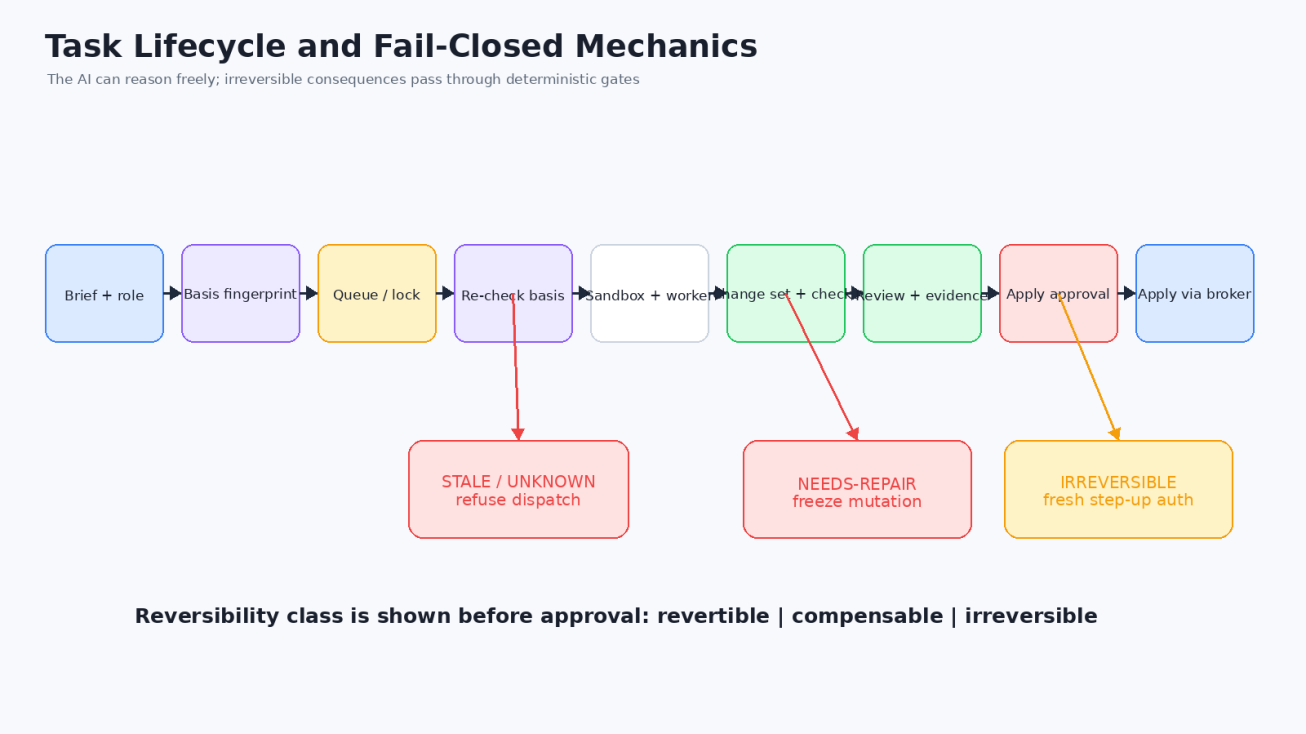
7.4 Context packs

Project files remain a convenient source, but the core concept is a named, versioned, hashable ContextPack. A context pack can contain repo files, owner-authored state files or other resources. Basis fingerprints record the exact resource_id + version values actually consulted.

08 Director and Mechanics Authority

8.1 Intelligence is not the security boundary

The Director may be an LLM early. Safety comes from mechanics that fail closed at stale basis, capability boundaries, exclusive locks, credential actions, repair states, reversibility and apply authorization. Planner determinism is not required for safety.



8.2 Director capabilities

Director may	Director may not
Propose/decompose tasks; select roles; prioritize; request checks/reviews; steer active work	Apply irreversible changes; grant itself capability; override stale/unknown basis; clear needs-repair; bypass exclusive locks; raise its own budget; access raw secrets

8.3 Event-driven supervision

The Director is woken only by meaningful events. A tokenless watcher/session layer handles idle time, liveness probes, terminal bell signals, approval timers and notification-delivery checks. Waiting must not consume model tokens merely to observe time passing.

8.4 Away mode

Irreversible or step-up-gated work is allowed to park indefinitely while the owner is away. The durable attention state is awaiting_user_verification, distinct from generic pending approval. Away digests include it, but no automatic timeout can convert absence into authorization.

- Batch normal escalations into a digest.
- Continue deterministic watchers while the Director sleeps.
- Escalate stuck notification delivery separately from an ordinary unanswered approval.

- Record delivery and acknowledgement state so the phone view distinguishes “waiting for you” from “you may never have received it.”

09 Agent Runtimes, Sessions and Supervision

9.1 Provider runtime seam remains provider-specific

Codex and Claude remain provider runtimes. Do not force a grand unified adapter too early. Keep a minimal boundary for start/cancel/send/status, then extract the richer contract only after both real integrations exist.

9.2 Session identity and resume are early contracts

Session identity is durable and auditable. Every session record stores provider_session_id, capture_method (native API | status-parse | operator-confirmed), captured_at and last_verified_at. Reattaching to an existing terminal process is distinct from resuming a provider conversation; they emit session.reattached and session.resumed separately. Regex/status capture is a fallback, never an unqualified truth claim.

9.3 Terminal bell as universal attention fallback

A provider with weak SDK events can still run in a durable terminal. BEL/terminal notification signals and prompt-state heuristics may emit agent.needs_input. Focusing or typing into the attached terminal can clear the marker when appropriate. This is provider-neutral and useful even when native events are rich.

9.4 Raw provider events

Raw provider payloads are never part of the durable product event stream. Debug captures are opt-in, short-retention files referenced by hash. Provider normalizers have golden replay fixtures.

10 Sandboxes, Exclusive Locks and Human Presence

10.1 Sandbox is the core term

A Sandbox is an isolated attempt workspace. Code may implement it with a git worktree. Assets may implement it with a serialized checkout or tool-specific copy. The core does not know how isolation works.

10.2 Exclusive resource locks

Heavy-native activities use named machine-wide locks. Queue waits trigger basis re-checks before acquisition and again immediately before mutation. If basis is unknown after a long wait, the task does not acquire/retain the lock: it requeues as blocked_basis_unknown and raises attention. If reconciliation becomes unknown while a lock is already held, the task enters needs-repair and the lock becomes held-by-frozen-task until the owner explicitly releases it.

10.3 Human claim / presence lease

Presence is explicit, not inferred from file mtimes. The owner can claim a workspace from the UI or CLI; tmux/Neovim hooks may heartbeat the claim. Dirty state always blocks destructive cleanup whether or not a claim is active.

```
workspace claim <workspace>
workspace heartbeat <workspace>
workspace release <workspace>
```

10.4 Teardown safety

- Never destroy a sandbox with uncommitted/unapplied work without an explicit recorded policy outcome.
- Before teardown, compute a SafetyRecord describing dirtiness, retained artifacts and reversal options.
- If reconciliation cannot prove ownership or safety, transition to needs-repair.

11 Remote Desk, Previews and Notifications

11.1 Remote Desk scope

The first user-facing product can be useful before multi-agent orchestration: mobile terminal attach, process/session status, read-only change-set view, local app preview, attention inbox, notifications and workspace claim controls.

11.2 Port broker and preview identity

The gateway allocates ports rather than assuming every dev server can own :3000. Preview routing is independent per preview; path-prefix proxying is avoided for root-assuming web apps, service workers and compressed Unity/WebGL assets.

11.3 Canonical browser origin

Tailscale is the preferred private network path; Cloudflare is a fallback path. Both must terminate at the same canonical browser origin/RP ID so WebAuthn critical-action verification behaves identically. Phase -1 proves DNS/TLS routing on a throwaway hostname before passkey enrollment.

11.4 Notifications

- Background WebSocket is not the primary phone notification channel.
- Use a reliable relay/channel such as ntfy, Telegram or Slack with a deep link into the canonical origin.
- Track requested -> queued -> delivered -> opened -> decided where available.
- A delivery watchdog distinguishes notification failure from owner delay.

12 Security, Credential Broker and Reversibility

12.1 Threat priorities

Threat	Primary control
Unlocked/stolen phone with live session	Short normal session + per-action step-up for critical consequences
Prompt injection / malicious repo content	Credential broker; capability limits; no raw secret injection by default
Stale task plan	Basis fingerprint and pre-dispatch re-check
Ambiguous recovery after crash	needs-repair + explicit repair operations
Approval fatigue	Attention metrics plus defect/rework guardrails
Mac host network exfiltration	Advisory egress only; secret minimization is the real boundary

12.2 Credential broker

The worker asks the gateway to perform privileged authenticated actions. Long-lived credentials remain in broker/host custody. Raw environment injection is an exception with elevated risk semantics.

12.3 Per-action user verification

Every reversal/compensation is itself an ApplyPlan. It is validated, budgeted and authorized like any other apply; if the reversal operation is irreversible or policy-sensitive, it requires fresh per-action verification. The broker executes it and emits apply.reversed or apply.compensated only after confirmed completion.

Critical operations require a fresh WebAuthn assertion bound to a specific action fingerprint. This includes every irreversible apply, privileged credential-broker action, security-policy change, budget increase and destructive filesystem operation.

```
action_fingerprint = hash({
  operation, project_id, task_id, basis_ref, change_set_hash,
  target_ref, apply_plan_hash, reversibility_class
})
```

12.4 Reversibility appears on approval

Approval UI shows risk tier and reversibility separately. A low-blast-radius action may still be irreversible; a high-blast-radius operation may be safely revertible. These are orthogonal review inputs.

13 Context Packs and Basis Fingerprints

13.1 Context packs

Canonical context is not limited to .workspace/*.md inside code repos. A ContextPack is a versioned set of resources that can come from workspace files or owner-level state. Roles declare which packs they may consult; each brief records which resources it actually used.

13.2 Staleness without false invalidation

Do not hash an entire context directory. Record only the resource_id/version pairs actually consulted. Editing an unrelated gotchas file does not stale a task that never read it. Changing a consulted architecture resource does.

13.3 Provider projection

The workspace owns canonical context. Provider-specific instruction files are generated projections, not the source of truth. This prevents drift when switching between Codex, Claude and terminal-oriented tools.

14 Data Model and Persistence

14.1 Core entities

Entity	Key fields / notes
Owner	id, display, budget ceilings, credential refs
Member	owner_id, kind human|agent, role_ref
Role	versioned charter/capabilities/context/output/limits/delegation
OrgNode	tree + policy overrides
Project	domain, adapter_binding, source_of_record, change_unit, change_budget, evidence_floor, exclusive_locks[], org_node_id
Task	role_id, basis, expected reversibility, execution class, acceptance criteria
Attempt	provider/session identity, session_capture_method, session_last_verified_at, sandbox_id, lifecycle
Sandbox	adapter, kind, safety_record_ref
ChangeSet	summary + RenderableChange refs + hash
Artifact	kind, path/ref, sha256, retention_class, owner task/attempt
CheckSpec / CheckResult	required/advisory, timeout, retry/flake policy; result, exit_code, artifact_ref, skipped_reason
Evidence	checks[] + profile + confidence lane + artifacts
Steer	intent, attempt, delivery_state, attempts, resolution
ReviewThread / ReviewComment	author identity, change_anchor, status, attempt linkage
Approval	risk, reversibility, reversal_plan_ref, action fingerprint, target_ref, apply_plan_hash
Preview	port, canonical route, health, owner task/attempt
Notification	channel, queued/delivered/opened/failed lifecycle, attention item
RepairCase	probes, affected resources, frozen locks, allowed ops, resolution
Lock	name, holder task/attempt, disposition active|held-by-frozen-task, acquired_at
BudgetReservation / BudgetLedger	owner/org scope, reserved, spent, released, ceiling

	window
HostRunner	host adapter binding + capabilities
ContextPack	named/versioned resource set

14.2 Identity columns from version 1

Every durable row and durable event carries owner_id and org_node_id. Even one-owner deployments use the same capability lookup path. This is a cheap seam now and an expensive migration later.

14.3 Storage rules

Artifacts use retention classes: transient (PTY/debug, hours-days), task-evidence (default 30 days or until task archive), milestone (manual retain), and build-cache (size/age LRU). Garbage collection never deletes an artifact referenced by an unresolved RepairCase, Approval, ReviewThread or retained milestone.

- SQLite + WAL for durable control-plane state.
- Schema migrations start at version 1.
- Large artifacts - screenshots, rendered changes, check logs, provider debug streams - live as files/object-like artifacts with hashes and references in SQLite.
- PTY bytes, token deltas and high-frequency progress ticks remain ephemeral.
- Raw provider debug streams are rotating, debug-only, short-retention artifacts.

14.4 needs-repair

needs-repair is read-only with respect to affected resources until the owner chooses Reprobe, Adopt, Reset/abandon, Release frozen lock, or an adapter-specific repair. If a task enters needs-repair while holding an exclusive lock, that lock becomes held-by-frozen-task and remains visibly separate from ordinary work. Releasing the lock is an explicit audited repair operation and does not resolve the RepairCase itself.

15 API, Realtime, Repair and Steering

15.1 Core API shape

```
GET  /api/office/state
GET  /api/projects
GET  /api/tasks/:id
POST /api/tasks
POST /api/tasks/:id/message
POST /api/tasks/:id/steers/:steer_id/retry
POST /api/tasks/:id/steers/:steer_id/mark-lost
POST /api/tasks/:id/steers/:steer_id/confirm-observed
POST /api/repair/:case_id/reprobe
POST /api/repair/:case_id/adopt
POST /api/repair/:case_id/reset
POST /api/repair/:case_id/release-lock
POST /api/reviews/:thread_id/comments
POST /api/reviews/:thread_id/request-changes
POST /api/approvals/:id/decide
POST /api/applies/:id/reverse-or-compensate
POST /api/projects/:id/claim
DELETE /api/projects/:id/claim
POST /api/previews
DELETE /api/previews/:id
```

15.2 Snapshot + durable events

Reconnect loads /api/office/state, then resumes the monotonic durable event stream from the snapshot sequence. Office state never depends on ephemeral PTY/token data. If sequence recovery is incomplete, fetch a fresh snapshot.

15.3 Steering delivery contract

POST /tasks/:id/message records a steer intent; it does not claim delivery. Delivery is a separate state: requested -> attempted -> acknowledged | failed | unresolved. An unresolved steer can block subsequent steering unless the owner explicitly marks it lost, confirms observation or retries.

15.4 Review feedback loop

Request changes persists comments/threads, bundles feedback into a review artifact, delivers it to the active or resumed worker session, and creates/links a new attempt. Rejection is not a dead end.

16 Evidence, Review Threads and Mobile Approval

16.1 Evidence profiles

CheckSpec includes timeout, max_retries, retry_backoff and flake_policy. Retries are bounded and recorded as separate CheckResult attempts. A flaky pass is not equivalent to a clean pass; it is surfaced in evidence and may downgrade the confidence lane according to Project policy.

Profile	Review-ready rule	Display/waiver behavior
strong	All required checks pass + reviewer complete	Skipped required check blocks unless owner waiver; never silently downgrade
partial	Required failures block; skipped required check requires explicit acknowledgement	Shows residual risk and missing evidence prominently
manual-required	Manual checklist + reviewer verification note required; automation is supporting evidence only	Never rendered as green-equivalent; owner acknowledgement required before apply

16.2 Review-readiness rule

Review-ready is a computed mechanics state, not a visual label. Each Project declares a profile floor and each CheckSpec is required or advisory. Strong: every required check must pass; a skipped required check blocks readiness unless the owner performs an explicit waiver with reason. Partial: failed required checks block; skipped required checks downgrade to manual acknowledgement before approval. Manual-required: automated checks may support review but can never upgrade the lane to green-equivalent; a mandatory manual checklist and reviewer “what I would verify” note are required. Only the owner can waive a required check, and the waiver is durable evidence.

16.3 Execution class

Class	Judgment delegated	Review implication
mechanical	Brief is decision-complete	Inspect implementation correctness
standard	Worker makes local implementation choices	Review changed logic and assumptions
deep	Worker chooses architecture/product behavior	Review decisions and implementation separately

16.4 Review threads

ReviewThread and ReviewComment are durable entities. Comments use domain-neutral change anchors: text_range, asset_id, node_path or region. Draft comments can be bundled and delivered to the worker as one feedback unit. Replies and resolution state remain scriptable and auditable.

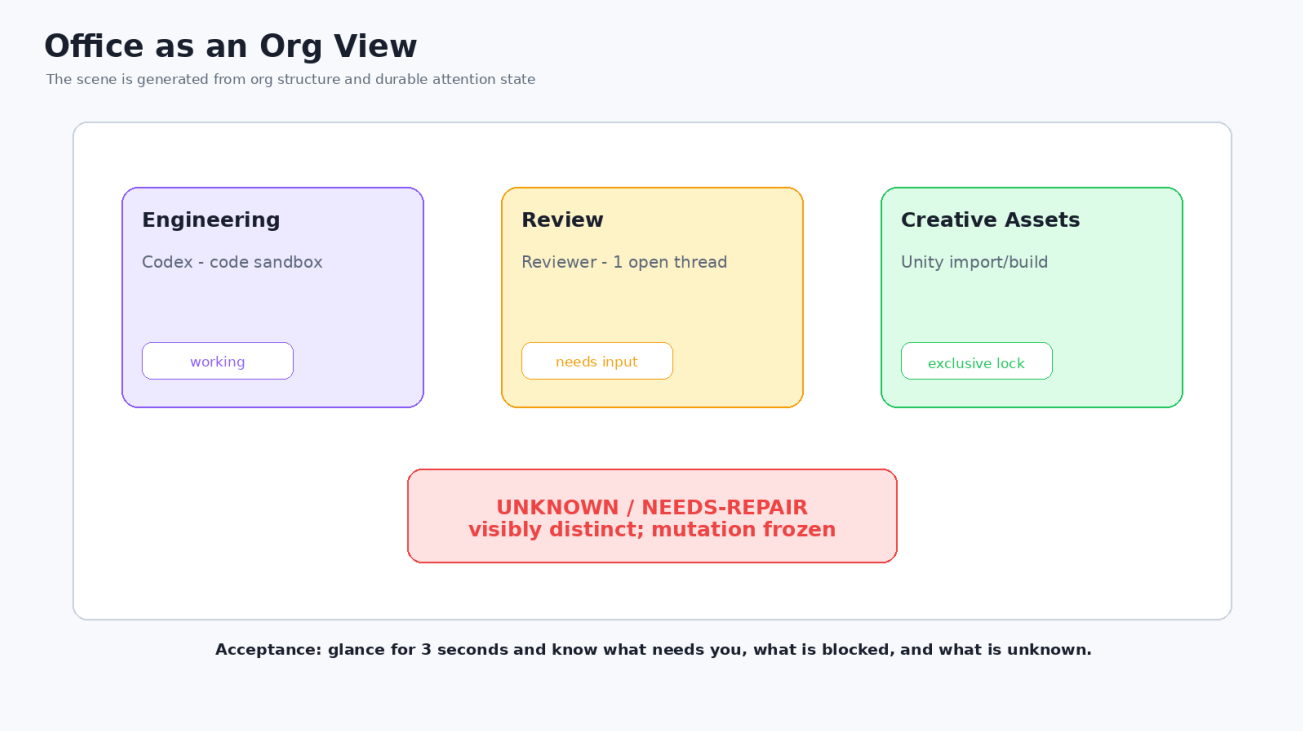
16.5 Mobile card

- Task summary + execution class + reversibility class at top.
- Evidence confidence and skipped checks are visible, not hidden in logs.
- File/asset-level triage: okay / look closer.
- Explain-this-anchor action invokes a fresh reviewer on that exact change anchor.
- Request changes preserves comments and starts a linked attempt.
- Apply requires step-up auth when policy/reversibility demands it.

16.6 Change budgets are domain-specific

Code may use lines/files. Heavy-native asset tasks may use assets, files or MB. The budget is a review-burden guardrail, not an arbitrary universal number.

17 Office as an Org View



17.1 Scene generation

- Scene structure derives from the OrgNode tree, not hard-coded rooms.
- Zone type derives from department/activity classes and current durable activity.
- Exclusive resource locks create visible shared facilities such as a Build Bay without special-case renderer logic.
- Attention badges derive from durable states: needs_input, review-ready, blocked, lock-held, needs-repair, unknown.

17.2 Truth rule

Never animate a lie

The office is presentation over snapshot + durable events. Ephemeral activity may decorate a known state, but it cannot create a durable fact. Unknown must look unknown.

17.3 Three-second glance test

From a phone, a three-second glance should answer: does anything need me, what is blocked, and is any state unknown/needs-repair? If a simple attention list beats the office, the office design failed.

18 Observability, Costs and Attention

18.1 Metrics

Metric family	Examples
Review efficiency	ready-to-land latency, review active time, comments per attempt
Quality guardrails	rework rate, rollback rate, escaped defects
Attention	approvals/task, unresolved notifications, needs_input age, away digest size
Mechanics reliability	repair frequency, stale-basis refusals, reconcile unknowns, session resume success
Budget	cost/task, cost/hour, cost/project, retry spend
Adapter health	check pass/fail/skip, sandbox open/close safety, apply/reversal outcomes

18.2 Cost controls

- Monthly, daily/hourly, task and run ceilings.
- Mid-run checks before expensive continuation, reviewer spawn or retry.
- Retry circuit breaker.
- When threshold crosses mid-operation: stop scheduling new work; interrupt safe activities; drain unsafe-to-interrupt activities; checkpoint and notify.

18.3 Attention watchdogs

- Approval requested but not delivered -> delivery incident.
- Delivered but not decided -> owner waiting state, not system failure.
- Unresolved steer -> visible blocker.
- Long queue behind exclusive lock -> basis re-check before execution.

19 Tech Stack and Implementation Choices

19.1 Recommended custom layer

Layer	Choice	Reason
UI	React + Vite + Tailwind	No SSR requirement; static bundle served by gateway; smallest runtime surface
Gateway/spine	TypeScript + Fastify/HTTP + WebSocket	Shared types with UI/protocol; good local tooling
Durable DB	SQLite + WAL	Single-host simplicity and inspectability
Terminal UI	xterm.js attaching to session manager	Browser renders terminal; persistence stays below gateway
Session manager	tmux first	Restart-proof practical default on macOS
Office renderer	PixiJS or Phaser, later	2D/isometric, phone-friendly, event-state mapping
Networking	Tailscale + canonical origin; Cloudflare fallback	Private default plus browser-only fallback
Host autostart	LaunchAgent in darwin adapter	Mac Mini user-session integration

19.2 Deliberately not portable

The UI framework, gateway language, database and office renderer are not seams. Changing them later may be a migration, and that is acceptable because they are not expected axes of product variation.

20 Testing and Contract Verification

20.1 Domain adapter contract tests

- Basis stale/fresh/unknown semantics.
- Sandbox close never destroys dirty work without a SafetyRecord/policy outcome.
- Change set hash stability and renderability.
- Declared check execution and explicit skipped_reason.
- Apply returns reversibility information and can be reconciled.
- needs-repair path is reachable and recoverable.

20.2 Two-adapter proof

Code and real binary assets must both satisfy the provisional contract. No core workaround is accepted for a method the asset adapter cannot implement. The contract is revised instead.

20.3 Fixture third adapter

Phase 7 adds a minimal text/docs adapter strictly as a portability regression fixture. It must compile and pass contract tests in under one day with no core changes. It is not a product feature.

20.4 Provider replay

Record sanitized provider event fixtures and replay them against normalizers. Raw captures remain files, not durable event rows. Provider SDK updates must fail tests rather than silently corrupt office state.

20.5 Host test double

HostAdapter has one real Darwin implementation and a test double. The core suite must run against the double, proving that launchd/tmux/TCC assumptions did not leak across the seam.

21 Repository Structure and Grep-able Rules

```
apps/
  web/
  gateway/
packages/
  domain/
  protocol/
  policy/
  persistence/
  attention/
adapters/
  domain/
    code/
    assets/        # provisional until Phase 4
  host/
    darwin/
  provider/
    codex/
    claude/
config/
  roles/
  org/
  context-packs/
state/
  artifacts/
  debug/
```

21.1 Vocabulary audits

Domain seam grep rule

No git, commit, branch, merge, worktree, diff or hunk identifier appears in packages/domain, packages/protocol or database schema. These words are allowed in adapters/domain/code where they are accurate.

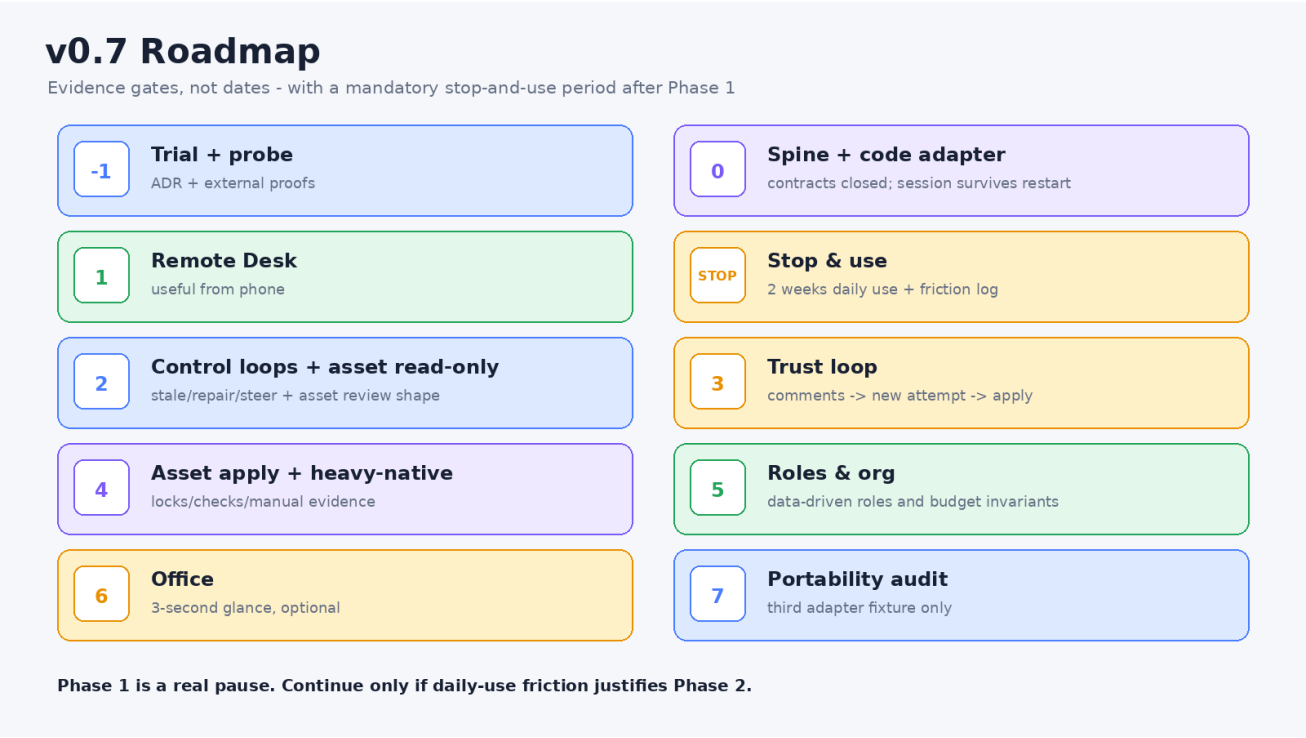
Host seam grep rule

No launchd, LaunchAgent, pmset, tmux, TCC, FileVault or ~/Library-specific symbol appears outside adapters/host/darwin.

21.2 Identity rule

No isMe/isOwner special-case authorization. owner_id and org_node_id are mandatory in durable records/events, and capability lookup is always the authorization primitive.

22 Roadmap



Phase	Size	Deliverable	Exit condition
-1 Trial + probe	M	Existing orchestrator trial; instrumented asset task; canonical-origin proof; naming clearance; resolve Phase-0 document gates	ADR fixes spine scope/delegation /asset shape; cleared repo/CLI/RP-ID names; check/render/apply-plan decisions recorded
0 Spine + code adapter	S-M	Protocol/domain packages; closed render union; code adapter; Darwin host adapter; persistent session manager; owner/org rows; migrations	Gateway restart while live session survives; vocabulary/identity audits pass; Phase-0 contract tests green
1 Remote Desk	M	Canonical origin; mobile terminal attach; bell->needs_input; read-only code change sets; preview; notifications; claim/lease	Useful from phone on real work
STOP & USE	-	Mandatory two-week daily-use period; continue friction log before Phase 2	Phase 2 scope is justified by observed

			friction, not blueprint momentum
2 Control loops + asset read-only slice	M-L	Basis staleness; session resume; steer/repair exits; asset snapshot/inspect/compute/render only	Real Unity asset change renders reviewably on phone; comments can anchor to asset/node/region; stale/unknown cases fail closed and recover
3 Trust loop	L	Review threads/comments; review-readiness; checks/evidence; fresh reviewer; broker; mobile apply; WebAuthn; reversibility	Request changes -> new attempt -> apply from phone with target-bound fingerprint and reversal plan
4 Asset apply + heavy-native	M	Asset apply plan/confirm; exclusive locks; import/build checks; manual-required evidence	One real asset task lands honestly; frozen-lock repair path works; SPI revised in writing if needed
5 Roles & org as data	M	Role templates, org tree, delegation invariants, budget reservations/sub-allocation, Director optional	Add role + sub-department without code change; no privilege/budget escalation
6 Office	M	Org-derived scene, attention badges, frozen/unknown distinct	Only if Phases 1-3 are useful daily; passes 3-second glance test
7 Portability audit	S	Minimal third domain adapter as test fixture only	Adapter written in <1 day; core untouched

Phase 1 is a deliberate stop-and-use point. Do not begin Phase 2 until two weeks of daily use produce a friction log that justifies it. Phase 7 belongs in the regression suite, not product scope.

23 Milestone Acceptance Gates

- Usable gate (end Phase 1): live terminal survives gateway restart; canonical mobile origin works; read-only change set is clearly marked “unchecked” before evidence exists; preview/notification/claim flows work from phone.
- MVP / trust-loop gate (end Phase 3): stale basis refuses dispatch; unresolved steer/repair is resolvable from phone; review comments drive a linked new attempt; review-readiness rules are enforced; target-bound action fingerprint + apply-plan hash are verified; irreversible apply requires fresh per-action verification.
- Quality guardrail: approval/attention improvements do not worsen rework, rollback, escaped defects or security exceptions.
- Portability audit (Phase 7): third fixture adapter compiles/passes contract tests in <1 day with no core edits; this is a test-suite property, not MVP scope.

24 Deferred Scope and Portability Invariants

Deferred	Carried later by	Invariant held now
Multi-tenancy / other owners	Seam C	owner_id on every row/event; no isMe branch
Hosted execution / remote runners	Seam B	All host calls behind HostAdapter; runner identity is data
Per-tenant credential custody	Credential broker	Broker takes owner scope today
Non-engineer onboarding	Org templates	Role/org config is versioned data
Template marketplace	Seam C	Templates have schema/version, not code
Multi-human review / ACLs	Seam C	Review authors have identity + author_kind
Other domains: docs/data/comms	Seam A	Domain contract and no git core vocabulary
Company attribution	Seam C	Audit records actor identity, not “the user”

Anti-scope principle

Deferring is not foreclosing. v0.7 pays for a seam only where a future change would otherwise require cross-cutting migration. It does not implement future users, runners or marketplaces.

25 Risks, Decisions and Anti-patterns

25.1 Key decisions now

- Single owner, one Mac Mini, one canonical browser origin.
- Three portability seams only (domain, host, owner/org); provider runtime is Seam D, a substitution seam deliberately not generalized until two integrations exist.
- Own thin cross-domain spine; code adapter may delegate to an existing orchestrator.
- Two real domain adapters before freezing the domain SPI.
- Persistent sessions live below the gateway.
- macOS egress is advisory; secret minimization and credential broker are the real boundary.
- LLM Director may appear early; deterministic mechanics govern consequences.

25.2 Anti-patterns

- Building tenancy, hosted runners or onboarding for users who do not exist.
- Freezing the domain contract on only code, then discovering at adapter three that it was git-shaped.
- Letting a code-oriented orchestrator become the source of truth for every domain.
- Naming core types after git operations and calling the design neutral.
- Grading blast radius without grading reversibility.
- Treating manual-required evidence as a temporary defect instead of a legitimate permanent state.
- Optimizing for fewer approvals by silently auto-approving riskier or irreversible actions.
- Building generic-first without a daily workload driving it.
- Keeping old calendar dates after deleting the old calendar scope.
- Animating state that cannot be reconstructed from durable truth.

Appendix A Durable / Ephemeral Event Vocabulary

A.1 Durable - persisted, sequenced, replayable

- project.registered
- task.created
- task.basis.captured
- task.stale
- task.dispatched
- attempt.started
- attempt.completed
- attempt.failed
- sandbox.opened
- sandbox.closed
- lock.acquired
- lock.released
- agent.needs_input
- steer.requested
- steer.acknowledged
- steer.failed
- steer.unresolved
- review.thread.created
- review.comment.added
- review.feedback.delivered
- evidence.ready
- approval.requested
- approval.decided
- apply.started
- apply.completed
- apply.failed
- repair.requested
- repair.resolved
- state.needs_repair
- task.completed
- task.cancelled

- agent.session.id_captured

- session.reattached

- session.resumed

- notification.queued

- notification.delivered

- notification.delivery_failed

- notification.opened

- budget.threshold
- budget.pause
- preview.started
- preview.healthy
- preview.failed
- preview.stopped
- project.claimed
- project.claim_released
- project.claim_expired
- artifact.created
- adapter.divergence
- apply.reversed
- apply.compensated
- repair.lock_released
- review.waiver_recorded

A.2 Ephemeral - memory/session channel only

- agent.message.delta
- process.output
- pty.binary
- progress.tick
- cursor/terminal paint state
- high-frequency tool progress

A.3 Event envelope

```
{
  seq, ts, type, owner_id, org_node_id, workspace_id?, task_id?, attempt_id?,
```

```
  actor: { member_id, role_id?, provider? },
  payload, artifact_refs[]
}
```

Appendix B Sample Configuration

B.1 Code Project

```
project:
  id: yes2portal
  domain: code
  adapter_binding: code.default
  source_of_record: /Users/.../yes2portal
  org_node: studio/engineering
  change_unit: lines
  change_budget: 250
  evidence_floor: strong
  checks:
    - {id: test, required: true, command: 'pnpm test', timeout_s: 600, max_retries: 1, flake_policy:
mark}
  exclusive_locks: []
```

B.2 Asset Project

```
project:
  id: nsr-assets
  domain: assets
  adapter_binding: assets.unity
  source_of_record: /Users/.../nsr
  org_node: studio/creative-assets
  change_unit: assets
  change_budget: 6
  evidence_floor: manual-required
  exclusive_locks: [unity-editor, unity-build]
```

B.3 Host capabilities

```
host:
  adapter: darwin
  session_manager: tmux
  capabilities:
    egress_enforcement: advisory
    persistent_sessions: true
    local_notifications: true
    biometric_step_up_via_webauthn: true
```

Appendix C Security and Go-Live Checklist

- ☐ Canonical browser origin resolves through private and fallback paths.
- ☐ Primary passkey enrolled and recovery/second authenticator procedure tested.
- ☐ Critical-action WebAuthn binds to action fingerprint.
- ☐ Credential broker holds privileged tokens; worker environments contain no unnecessary raw secrets.
- ☐ macOS egress described as advisory in UI/docs.
- ☐ Allowed workspace roots and path canonicalization tested.
- ☐ Session-manager restart/reattach proof passed.
- ☐ Gateway restart does not kill live agent session.
- ☐ needs-repair and steer-resolution operations work from phone.
- ☐ Irreversible action test requires fresh step-up and displays no-reversal warning.
- ☐ Cost watchdog thresholds and retry circuit breaker configured with real values.
- ☐ Backups cover SQLite plus referenced artifacts and configuration.
- ☐ FileVault/power-loss availability limitation accepted.
- ☐ TCC/permissions health visible on dashboard.

Appendix D Gates and Decisions by Phase

Gate	Decision / v0.7 answer
Phase 0 - Render protocol	Closed union in packages/protocol. Adapters produce known render/anchor shapes; no adapter-supplied browser code in v0.7.
Phase 0 - Apply boundary	Use apply_plan + spine-owned broker execution + confirm_applied. Adapters never receive broker handles.
Phase 0 - Config precedence	Project/domain owns change_unit. Most-restrictive-wins for budget/evidence; looser role config is load-time error.
Phase 0 - Check spec	Domain-neutral CheckSpec: id, required/advisory, command/adapter_op, timeout, max_retries, flake_policy, expected artifacts.
Phase -1 output - Naming	Still external: clear product/repo/CLI/hostname/RP ID before Phase 0 repository initialization. Product document uses descriptive title; core entity is Project to avoid collision.
Phase 1 - Canonical origin	Must be proven over both private and fallback paths on a throwaway hostname before passkey enrollment.
Phase 1 - Passkey bootstrap/recovery	First enrollment only from local physical-console session or one-time setup token shown locally; adding authenticators requires existing fresh WebAuthn; keep a second authenticator; lost-device recovery requires physical-console/local reset and invalidates old credentials.
Phase 1 - Real cost ceilings	Owner must set monthly/hourly/task/run values before automation is enabled; placeholders are not executable configuration.
Phase 3 - Review readiness	Defined in 16.2; required-check waivers are explicit owner-authored durable evidence.
Phase 3 - Reversibility taxonomy	revertible / compensable / irreversible; per-operation class in ApplyPlan, with target and reversal plan shown before approval.
Phase 4 - Asset adapter granularity	Provisional choice: one assets adapter with per-tool capability probes; split only if Unity/Defold/Blender semantics cannot share the contract.
Phase -1/4 - Code delegation	Phase -1 ADR decides whether external orchestrator owns sandbox/session mechanics; if delegated, adapter.divergence is mandatory.

Appendix E Source and Competitive Notes

These sources are references for Phase -1 evaluation and mechanism comparison; they are not dependencies by default.

- https://github.com/atqamz/hand - coding-agent fleet/orchestration reference; evaluate as a code-adapter substrate.
- https://github.com/kunchenguid/firstmate - verified current repository for event-driven supervision, persistent session backends, guarded operations and away-mode patterns.
- https://devhq.app/docs/worktrees - worktree-oriented workflow reference supplied during v0.6 review.
- https://devhq.app/docs/agents - agent terminal/session attention reference supplied during v0.6 review.
- https://devhq.app/docs/review - inline review-thread workflow reference supplied during v0.6 review.
- https://tailscale.com - private network and service exposure reference.
- https://developers.cloudflare.com/cloudflare-one/ - fallback Access/Tunnel reference.
- https://www.w3.org/TR/webauthn-3/ - WebAuthn model for per-action user verification.

Appendix F v0.7 Delta from v0.6

v0.6 position	v0.7 execution-baseline change
Renderable undefined / anchor shapes leaked implicitly	Closed versioned render/anchor union is explicit in protocol
Adapter apply held broker handle	Declarative ApplyPlan; spine validates and broker-executes
No pre-teardown dirtiness probe	inspect_sandbox added and drives safety/office state
needs-repair could freeze lock forever without modeled exit	Frozen-lock disposition + explicit release operation + blocked-queue attention
Project/Role policy precedence ambiguous	Project safety floor + most-restrictive-wins; invalid loosening fails at load
Evidence profiles lacked readiness semantics	Review-readiness rules, required/advisory checks and durable waivers defined
Data/event model omitted load-bearing records	Steer, Artifact, Preview, Check*, Notification, RepairCase, Lock, Budget ledger + missing durable events added
Provider seam contradicted three-seam rule	Provider named as Seam D substitution boundary, not portability seam
Asset proof arrived after trust UI	Read-only asset render slice moved into Phase 2 before trust loop
No build sizing or stop point	S/M/L phase bands + mandatory two-week stop-and-use after Phase 1
MVP mixed all phases	Split into Usable, MVP/trust-loop and Portability-audit gates

Closing Design Test

Daily-use test

Be away from the machine, glance at the phone, know whether anything needs you or is unknown, trust that stale or irreversible work cannot silently apply, and confidently review one small unit of agent work.

Portability test

When the next domain, host or owner becomes real, it arrives as a new adapter/module plus configuration - not as a migration through the whole spine. If a proposed feature serves neither this test nor the daily-use test, it is Phase 8 or later.

End of v0.7 blueprint.